

VitaCore
Dev

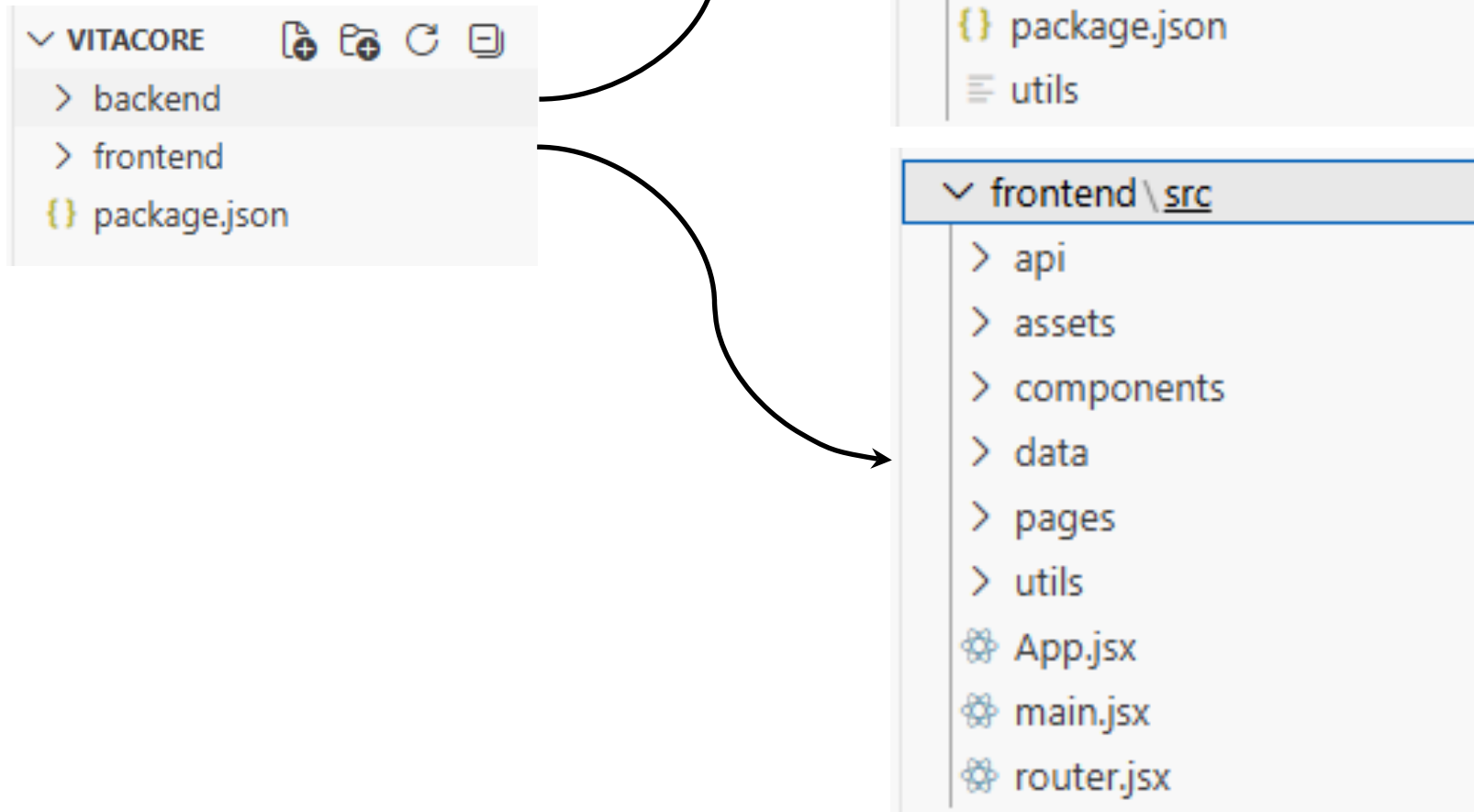
목차

1. 파일 구조 분리

2. 핵심 코드 설명

3. 현재 개발 중 및 향후 계획

1. 파일 구조 분리



1. 파일 구조 분리 - BACKEND

```

  backend
  |
  |> node_modules
  |
  |> src
  |   |
  |   |> config
  |   |   |
  |   |   |JS db.js
  |   |   |JS mailer.js
  |   |   |
  |   |   > controllers
  |   |   > middlewares
  |   |   > routes
  |   |   > services
  |   |   > templates
  |   |       |
  |   |       |JS emailTemplates.js
  |   |       > utils
  |   |       > validators
  |   |
  |   |JS app.js
  |   |JS server.js

```

- **config:** DB 및 메일 설정 관리
- **controllers:** 요청/응답 처리
- **middlewares:** JWT 인증 및 에러 처리
- **routes:** API 엔드포인트 분리
- **services:** 핵심 비즈니스 로직 처리
- **templates:** 이메일 템플릿 관리
- **utils:** 공통 함수 및 DB 유틸 관리
- **validators:** 입력값 검증

1. 파일 구조 분리 - POSTMAN 활용

REQUEST HISTORY		...
▼ March 30		
POST	http://localhost:3000/api/auth/login	
POST	http://localhost:3000/api/auth/login	
POST	http://localhost:3000/api/auth/signup	
POST	http://localhost:3000/api/auth/signup	
POST	http://localhost:3000/api/connection-...	
POST	http://localhost:3000/api/auth/login	
POST	http://localhost:3000/api/auth/login	
POST	http://localhost:3000/api/auth/login	
▼ March 29		
POST	http://localhost:3000/api/auth/resend...	
POST	http://localhost:3000/api/auth/resend...	
POST	/api/auth/resend-verification	
POST	http://localhost:3000/api/auth/signup	
▼ March 28		
POST	http://localhost:3000/characters	
POST	http://localhost:3000/characters	
POST	http://localhost:3000/login	
POST	http://localhost:3000/login	
▼ March 27		
POST	http://localhost:3000/login	
POST	http://localhost:3000/signup	
POST	http://localhost:3000/signup	

로그인 및 회원가입 기능의
명료성과 안전성을 보장하기 위한 테스트
-> 시간이 지나며 인증 구조가 확립되었음

1. 파일 구조 분리 - 이메일 인증

☆ [VitaCore] 이메일 인증을 다시 진행해주세요 

^ 보낸사람 VitaCore <vitacore.service@gmail.com>

받는사람 asdfg041231@naver.com

2026년 3월 29일 (일) 오전 3:33

이메일 인증 시 버튼 인증을 시도하였으나,
링크 클릭 인증 방식으로 변경하였음

VitaCore 이메일 인증 재발송

안녕하세요, 고은채님.

아래 버튼을 눌러 이메일 인증을 완료해주세요.

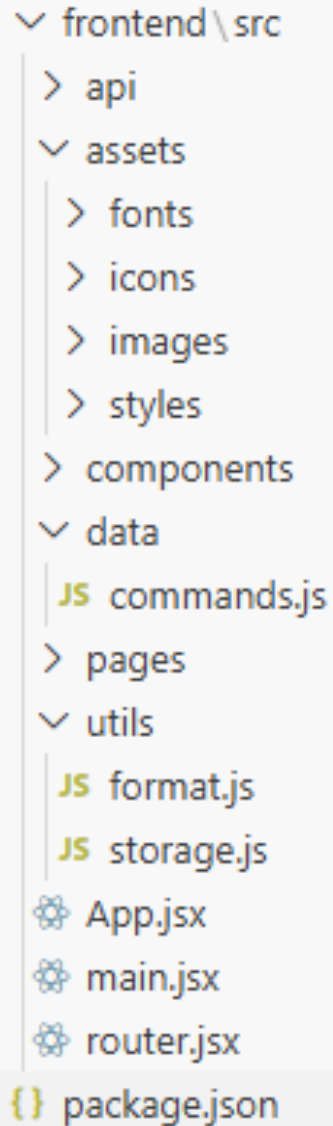
이메일 인증하기

버튼이 작동하지 않으면 아래 링크를 브라우저에 붙여넣어 주세요.

<http://localhost:3000/api/auth/verify-email?token=799b5d32503a217c8d1e2c9b8fad3af62d5f2c2e9390c0ab7ae74a2623350d09>

이 링크는 30분 동안 유효합니다.

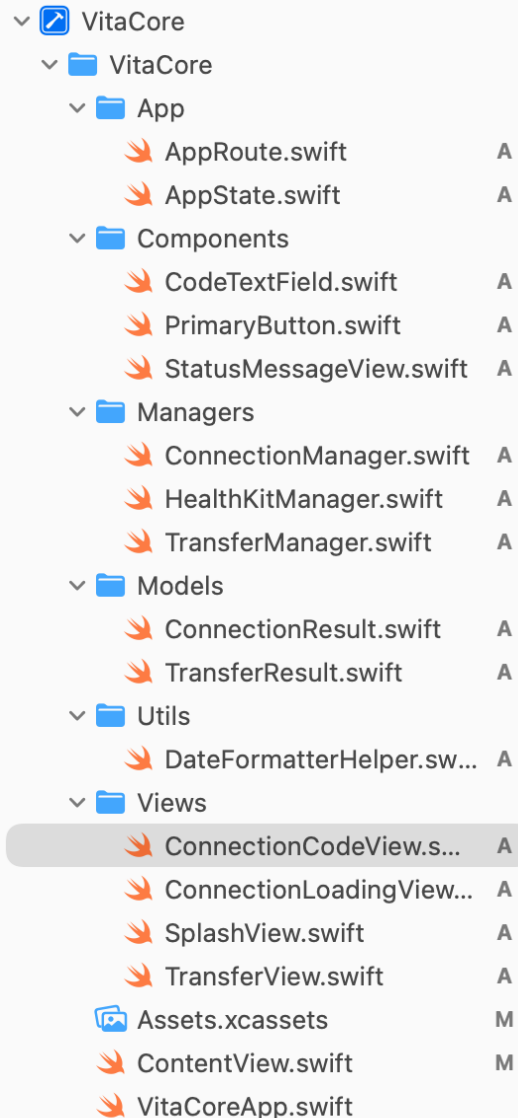
1. 파일 구조 분리 - FRONTEND



```
▼ frontend \ src
  > api
  ▼ assets
    > fonts
    > icons
    > images
    > styles
  > components
  ▼ data
    JS commands.js
  > pages
  ▼ utils
    JS format.js
    JS storage.js
  ⚙ App.jsx
  ⚙ main.jsx
  ⚙ router.jsx
  {} package.json
```

- **api:** 백엔드 API 요청 관리
- **assets:** 폰트, 아이콘, 이미지, CSS 등
- **components:** 공통 UI 컴포넌트 관리
- **data:** 명령어 및 고정 데이터 관리
- **pages:** 화면 단위 페이지 구성
- **utils:** 포매팅 및 로컬 저장소 공통 기능 관리
- **App.jsx:** 전체 앱 구조 관리
- **main.jsx:** React 앱 실행 진입점
- **router.jsx:** 페이지 라우팅 관리

1. 파일 구조 분리 - iOS



- **App:** 앱 전체 상태 및 화면 흐름 관리
- **Components:** 공통 UI 컴포넌트 관리
- **Managers:** 연결, HealthKit, 데이터 전송 로직 처리
- **Models:** 결과 데이터 및 상태 모델 관리
- **Utils:** 날짜 포맷 등 공통 기능 관리
- **Views:** 실제 화면 구성
- **ContentView.swift:** 앱 실행 시 기본 화면
- **VitaCoreApp.swift:** iOS 앱의 시작 지점

2. 핵심 코드 설명 - JWT 인증

관련 파일

- `// backend/src/services/authService.js`
- `// backend/src/middlewares/authMiddleware.js`

구현 내용

- 로그인 시 JWT 발급
- 인증 필요한 API 접근 시 토큰 검증
- middleware 기반 인증 처리 구조 적용

```
jwt.verify(token, JWT_SECRET, (err, decoded) => {
```

2. 핵심 코드 설명 - Connection Code

관련 파일

- `// backend/src/services/connectionCodeService.js`
- `// VitaCore/Managers/ConnectionManager.swift`

```
const expiresAt = new Date(Date.now() + 5 * 60 * 1000);
```

구현 내용

- 캐릭터 기반 Connection Code 생성
- iOS 앱에서 코드 입력 후 서버 검증
- 연결 성공 시 기기 등록 처리

2. 핵심 코드 설명 - HealthKit 데이터 전송

관련 파일

- `// backend/src/services/measurementService.js`
- `// VitaCore/Managers/TransferManager.swift`
- `// VitaCore/Managers/HealthKitManager.swift`

구현 내용

- Apple HealthKit 데이터 읽기
- VitaCore 서버로 데이터 전송
- measurement 테이블 저장 처리

```
const insertResult = await dbQuery(insertMeasurementSql, [  
  device.character_id,  
  device.id,  
  vitalTypeRow.id,  
  value,  
]);
```

3. 현재 개발 중 및 향후 계획

- Frontend - Backend API 연동 진행
- Connection Code 무결성 체크
- iOS Relay App 안정화 및 HealthKit 연동 보완
- Vital Monitor 시각화 기능 개발
- Measurement 이력 및 이상값 처리 고도화
- 보안 로그 및 예외 처리 구조 개선
- 최종 테스트 및 프로젝트 통합 진행 예정(+ 리팩토링)